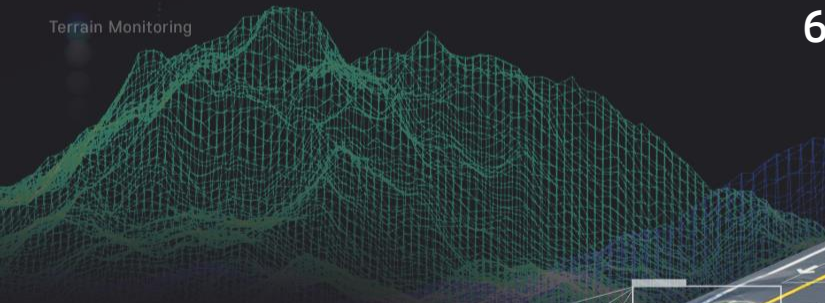


Image Recognition



Terrain Monitoring



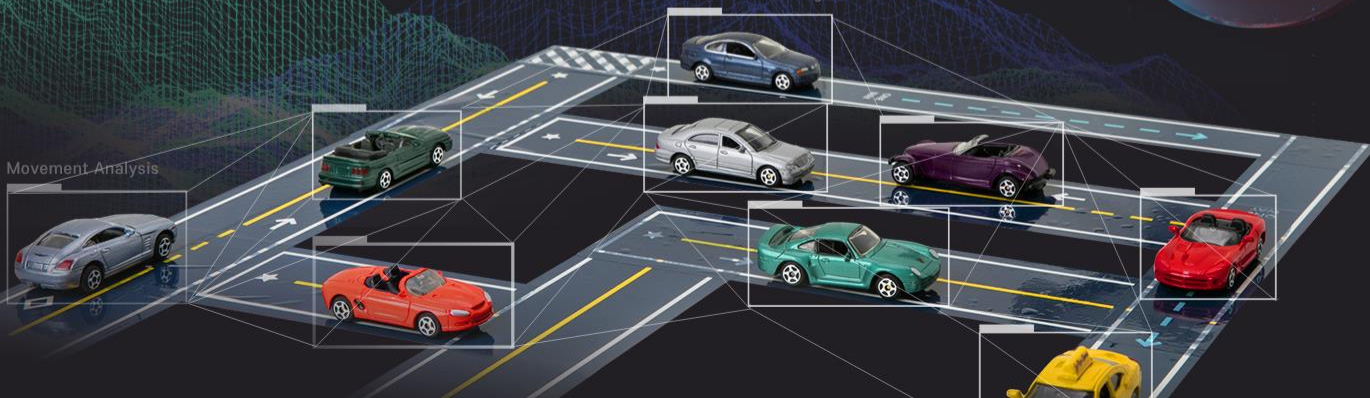
강원혁신플랫폼

컴퓨터 비전

6개의 클래스 분류

Autonomous Driving

Movement Analysis



권장

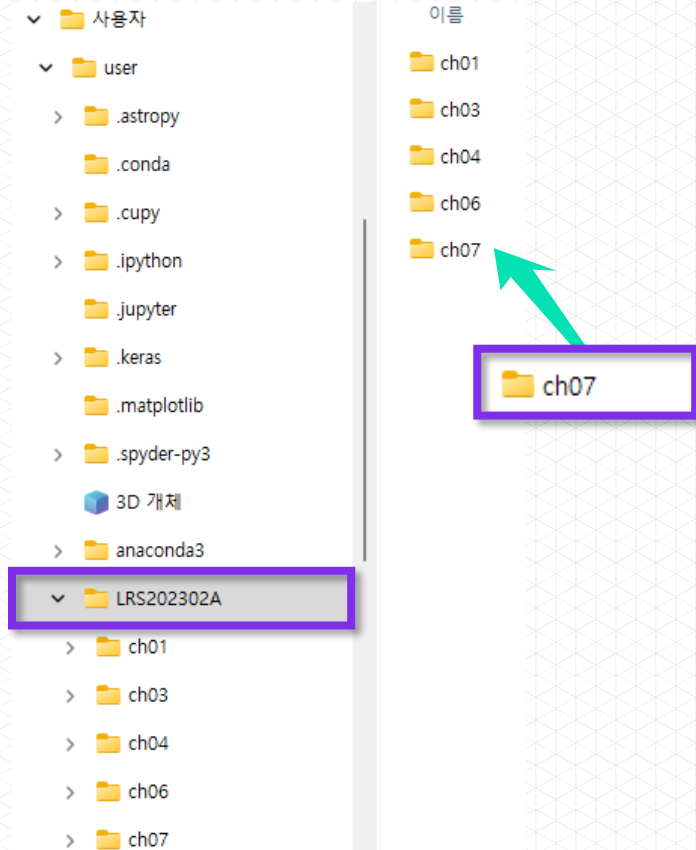
아래와 같은 경로에 실행 소스가 존재하면
환경 구축 완료

c:\Users\user>LRS202302A>

컴퓨터 이름

또는

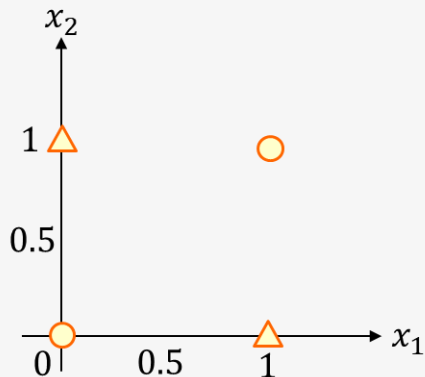
사용자 계정



- XOR 게이트의 경우 OR 게이트 때처럼 $\bigcirc(=0)$ 과 $\triangle(=1)$ 으로 출력을 나타내면 아래 그림과 같음

직선 하나로 $\bigcirc(=0)$ 과 $\triangle(=1)$ 을 나누는 영역을 만들 수 있을까?

결론부터 말하자면, 직선 하나로 나누기란 불가능함



XOR 게이트 출력($\bigcirc=0$, $\triangle=1$)

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

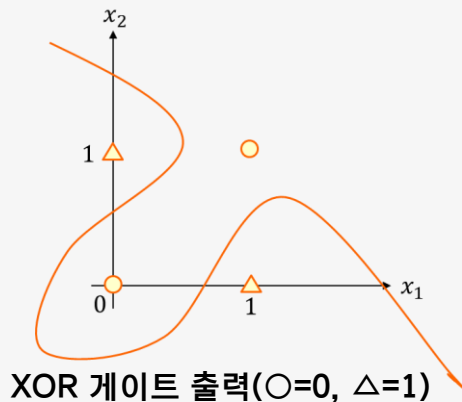
XOR 게이트의 진리표

■ 하지만, 아래 그림과 같이 ○(=0)과 △(=1)을 곡선으로 나눌 수 있음

⦿ 퍼셉트론은 직선 하나로 나눈 영역만 표현할 수 있다는 한계가 있음

⦿ 즉, 아래의 그림과 같이 곡선은 표현할 수 없음

아래 그림과 같은 곡선의 영역을 비선형 영역, 직선의 영역을 선형 영역이라고 함

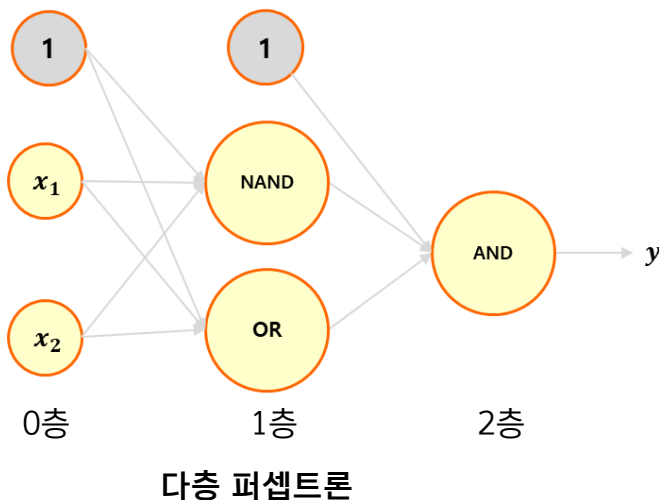
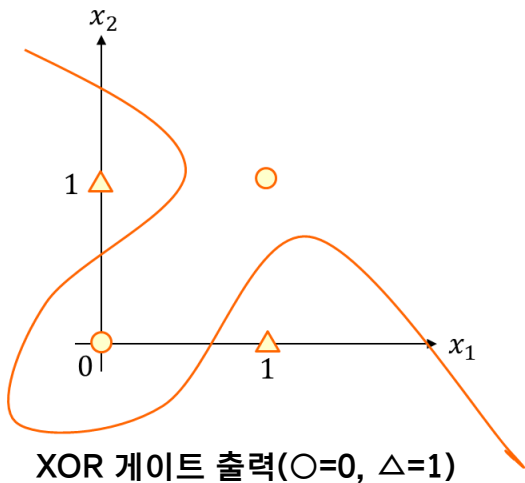


x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

XOR 게이트의 진리표

■ 퍼셉트론으로는 XOR 게이트를 표현할 수 없음

- 하지만 퍼셉트론은 '층을 쌓아' 다층 퍼셉트론(Multi-layer perceptron)을 만들 수 있음
- 즉, 아래의 그림과 같이 퍼셉트론을 조합(층을 쌓아서)하여 XOR 게이트를 구현할 수 있음



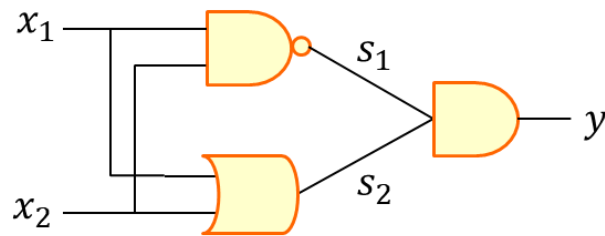
x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

XOR 게이트의 진리표

다음은 AND, OR, NAND 게이트를 사용하여 XOR 게이트를 구현하는 코드이다.

실행결과 잘 동작되는 것을 볼 수 있음

```
def XOR(x1, x2):  
    s1 = NAND(x1, x2)  
    s2 = OR(x1, x2)  
    y = AND(s1, s2)  
    return y  
  
if __name__ == '__main__':  
    for xs in [(0, 0), (0, 1), (1, 0), (1, 1)]:  
        y = XOR(xs[0], xs[1])  
        print(str(xs) + " -> " + str(y))  
#(0, 0) -> 0  
#(1, 0) -> 1  
#(0, 1) -> 1  
#(1, 1) -> 0
```



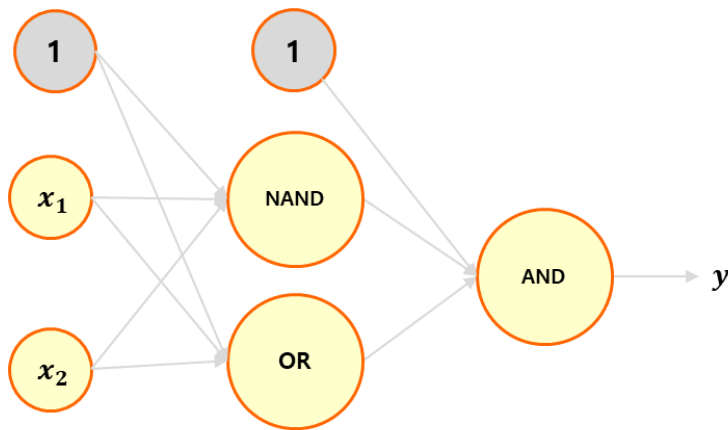
XOR 게이트

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

XOR 게이트의 진리표

■ 단층 퍼셉트론으로 표현하지 못한 것을 층을 쌓아 구현할 수 있는 것을 알 수 있음

- ➡ 퍼셉트론은 층을 쌓아(딥러닝) 더 다양한 것을 표현할 수 있음
- ➡ 앞의 XOR 신경망의 가중치와 편향은 사람이 설정하였지만, 데이터에서 학습하여 설정할 수 있음
- ➡ 즉, 알고리즘이 데이터 특징을 학습하여 답안지(가중치, 편향)를 작성할 수 있음



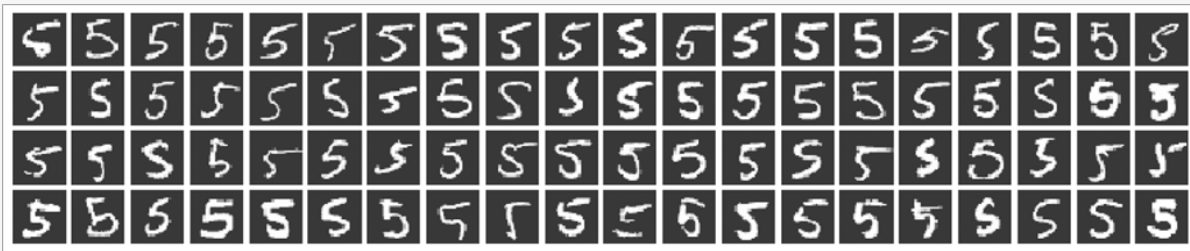
XOR 신경망



■ 다음의 문제를 어떻게 풀 수 있는지 생각해 보자.

- 아래 그림의 이미지에서 숫자 '5'를 인식하는 프로그램을 구현하는 것이 목표임
- 머릿속에 떠오르는 프로그램을 구현하기 위한 아이디어(알고리즘)는?
- 사람은 어렵지 않게 인식하지만, 그 안에 숨은 규칙성을 명확한 로직으로 풀기가 만만치 않음

숫자 '5'를 인식하는 알고리즘을 주어진 데이터를 잘 활용해서 해결할 수 있다면 어떨까?



손글씨 숫자 '5'의 예

활성화 함수(Activation function)

입력 신호의 총합을 출력 신호로 변환하는 함수(앞에서 설명한 $h(x)$ 함수).
입력 신호의 총합이 활성화를 일으키는지를 정하는 역할을 함.

- ⦿ 아래의 좌측 식은 가중치가 곱해진 입력 신호의 총합을 계산하고, 그 합을 활성화 함수에 입력해 결과를 내는 2단계로 처리됨
- ⦿ 그래서 이 식을 우측과 같이 2개의 식으로 나눌 수 있음

$$y = h(b + w_1x_1 + w_2x_2)$$

$$h(x) = \begin{cases} 0 & (x \leq 0) \\ 1 & (x > 0) \end{cases}$$

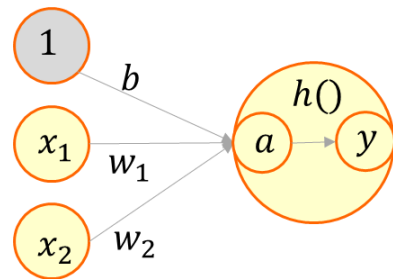
퍼셉트론의 식



$$a = b + w_1x_1 + w_2x_2$$

$$y = h(a)$$

아래의 오른쪽 식을 뉴런 그림으로 나타내면 다음과 같음



활성화 함수의 처리 과정

$$y = h(b + w_1x_1 + w_2x_2)$$

$$h(x) = \begin{cases} 0 & (x \leq 0) \\ 1 & (x > 0) \end{cases}$$

퍼셉트론의 식

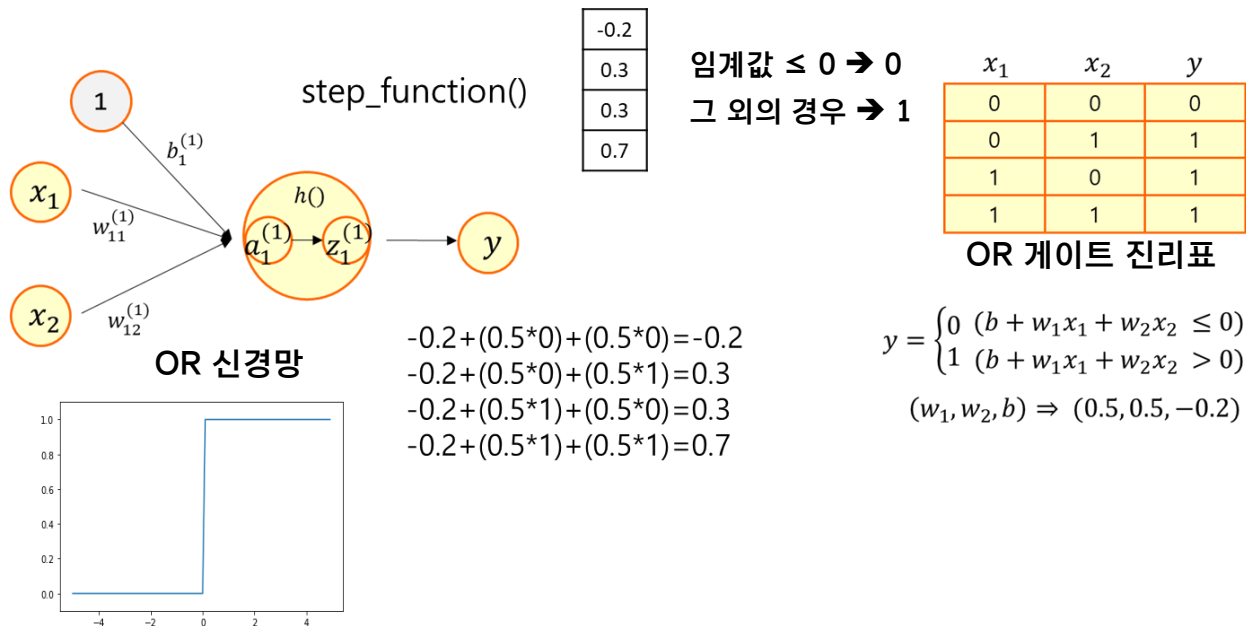


$$\begin{aligned} a &= b + w_1x_1 + w_2x_2 \\ y &= h(a) \end{aligned}$$

- ⊕ 위 식 오른쪽에서 **a**는 가중치가 달린 입력 신호와 편향의 총합을 계산한 것
- ⊕ **y**는 **a**를 **h()**함수에 넣어 출력하는 흐름

퍼셉트론에서 신경망(딥러닝)으로 가기 위한 첫걸음(길잡이)

- 일반적으로 단층 퍼셉트론은 단층 네트워크에서 계단 함수(임계값을 경계로 출력이 바뀌는 함수)를 활성화 함수로 사용한 모델임



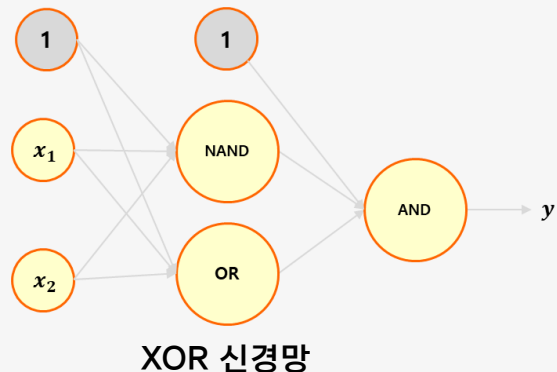


■ 퍼셉트론에서 신경망(딥러닝)으로 가기 위한 첫걸음(길잡이)

- 다층 퍼셉트론은 신경망(여러 층으로 구성되고 시그모이드 함수 등의 매끈한 활성화 함수를 사용하는 네트워크)을 의미함

XOR 신경망 네트워크

이 신경망에 활성화 함수로 계단 함수와 시그모이드 함수를 사용하는 경우를 생각해보자.



- 계단 함수인 경우 0과 1 중 하나의 값을 다음 뉴런으로 전달할 것임
- 시그모이드 함수인 경우 실수(0.731..., 0.880... 등)값을 다음 뉴런으로 전달할 것임

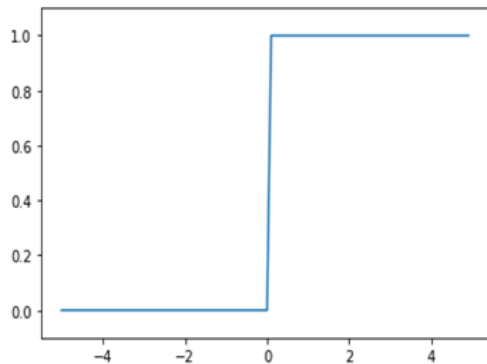


■ 퍼셉트론에서 신경망(딥러닝)으로 가기 위한 첫걸음(길잡이)

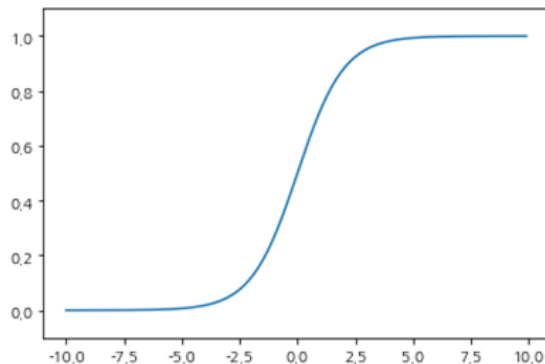
🔄 정리해 보면 다음과 같음

1 퍼셉트론에서는 뉴런 사이에 0 혹은 1이 흘렀음

2 신경망에서는 연속적인 실수(0.731..., 0.880... 등)가 흐름



계단 함수



시그모이드 함수



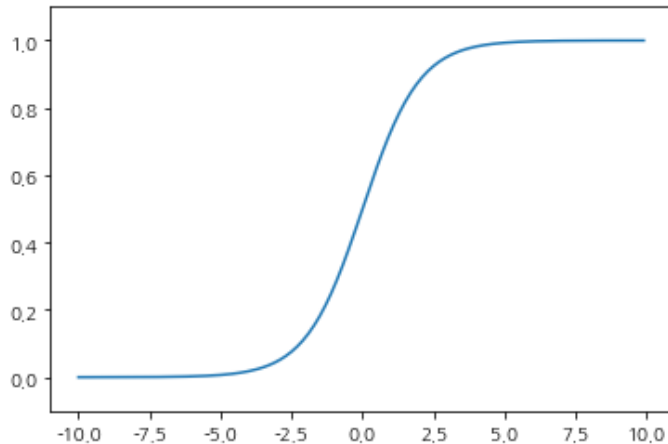
시그모이드 함수



- 신경망에서 자주 이용되는 활성화 함수인 시그모이드 함수(Sigmoid function)를 나타내는 식

$$h(x) = \frac{1}{1 + \exp(-x)}$$

- ② 위 식에서 $\exp(-x)$ 는 e^{-x} 를 뜻하며, e 는 자연상수로 2.7182...의 값을 갖는 실수임
- ② 위 식에서 알 수 있는 것처럼, 시그모이드 함수는 입력을 주면 출력을 돌려주는 변환기임



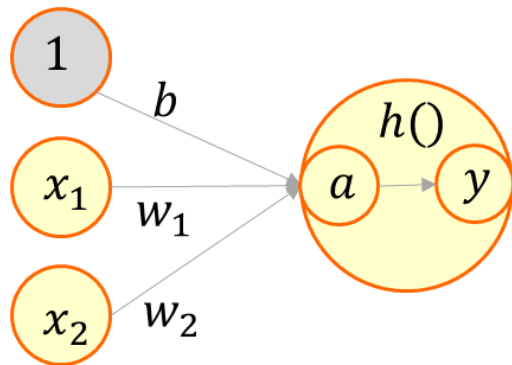


시그모이드 함수



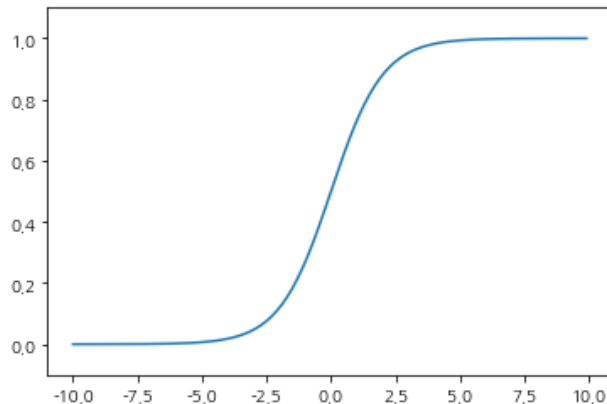
- 신경망에서 활성화 함수로 시그모이드 함수를 이용하면, 어떠한 값이 들어와도 0~1 사이 실수값이 다음 뉴런에 전달됨

즉, 0 ~ 1의 실수 값이 다음 뉴런에 전달됨



활성화 함수의 처리 과정

$$h(x) = \frac{1}{1 + \exp(-x)}$$



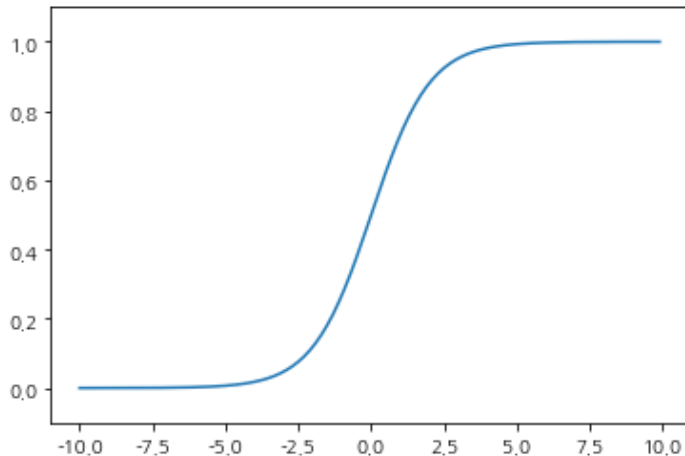


시그모이드 함수



■ 0에서 1 사이의 값을 출력함

```
def sigmoid(x):  
    return 1 / (1 + np.exp(-x))  
  
x = 1.0  
y = sigmoid(x)  
print(y)    # 0.7310585786300049  
  
x = -1.0  
y = sigmoid(x)  
print(y)    # 0.2689414213699951
```





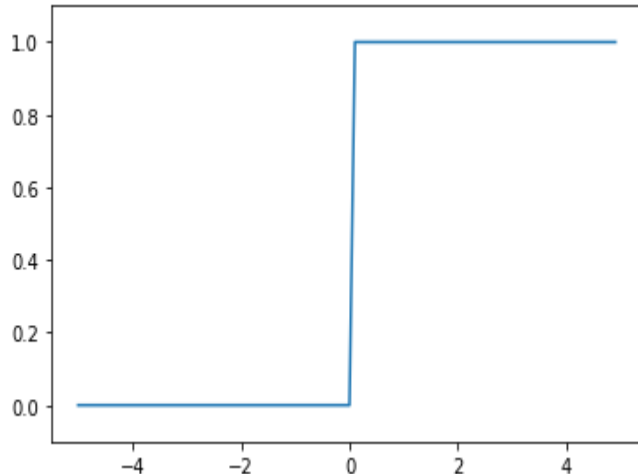
계단 함수



- 계단 함수(Step function)는 아래의 식과 같이 입력(>0)이 0을 넘으면 1을 출력하고, 그 외에는 0을 출력하는 함수

$$y = h(b + w_1x_1 + w_2x_2)$$

$$h(x) = \begin{cases} 0 & (x \leq 0) \\ 1 & (x > 0) \end{cases}$$



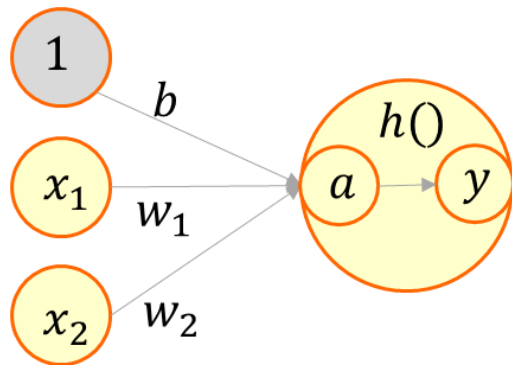


계단 함수



- 신경망에서 활성화 함수로 계단 함수를 이용하면, 입력(>0)이 0을 넘으면 1의 값, 그 외에는 0의 값을 다음 뉴런에 전달함

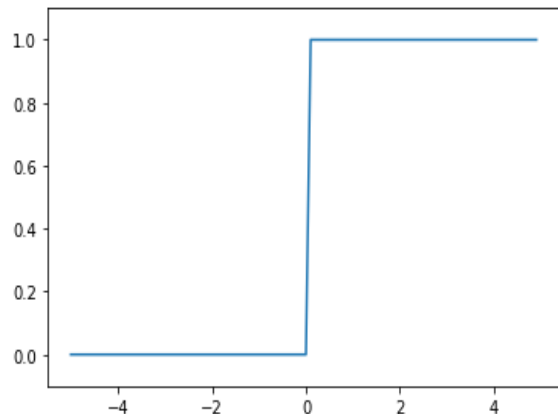
즉, 0 또는 1의 값이 다음 뉴런에 전달됨



활성화 함수의 처리 과정

$$y = h(b + w_1x_1 + w_2x_2)$$

$$h(x) = \begin{cases} 0 & (x \leq 0) \\ 1 & (x > 0) \end{cases}$$





계단 함수

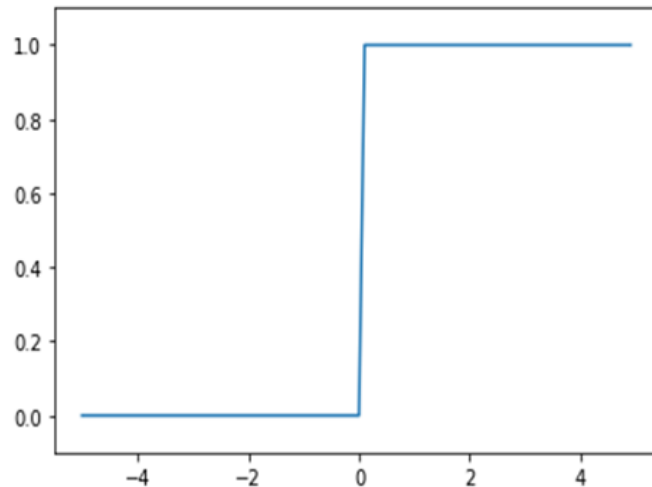


■ 0 또는 1의 값을 출력함

```
def step_function(x):  
    if x > 0:  
        return 1  
    else:  
        return 0
```

```
x = 1.0  
y = step_function(x)  
print(y) #1
```

```
x = -1.0  
y = step_function(x)  
print(y) #0
```



계단 함수

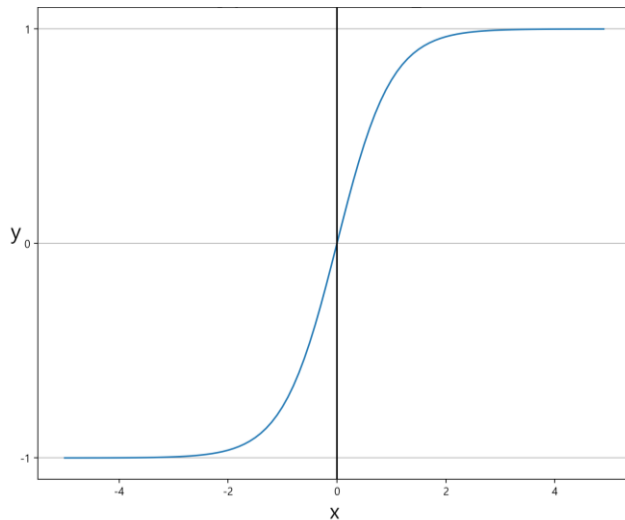


하이퍼볼릭 탄젠트 함수 🔍

- 하이퍼볼릭 탄젠트(Hyperbolic tangent, tanh) 활성화 함수는 **시그모이드 함수를 선형변환 한 것으로 시그모이드의 단점(그래디언트 소실)을 보완**해주는 활성화 함수

$$h(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

- ② 위 식에서 e^x 는 $\exp(x)$ 를 뜻하며, e 는 자연상수로 2.7182...의 값을 갖는 실수임
- ② tanh 함수는 **-1에서 1 사이의 값을 출력**하고, 중앙값은 0임

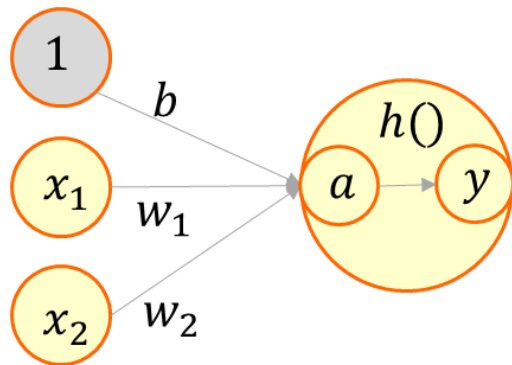




하이퍼볼릭 탄젠트 함수 🔍

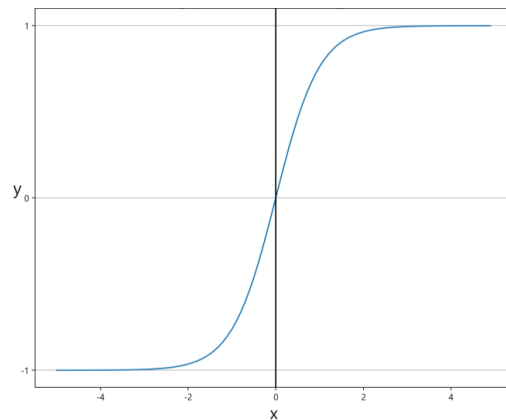
- 신경망에서 활성화 함수로 **tanh** 함수를 이용하면, 어떠한 값이 들어와도 **-1~1** 사이 실수값이 다음 뉴런에 전달됨

🔄 즉, **-1 ~ 1** 사이의 실수값이 다음 뉴런에 전달됨



활성화 함수의 처리 과정

$$h(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$





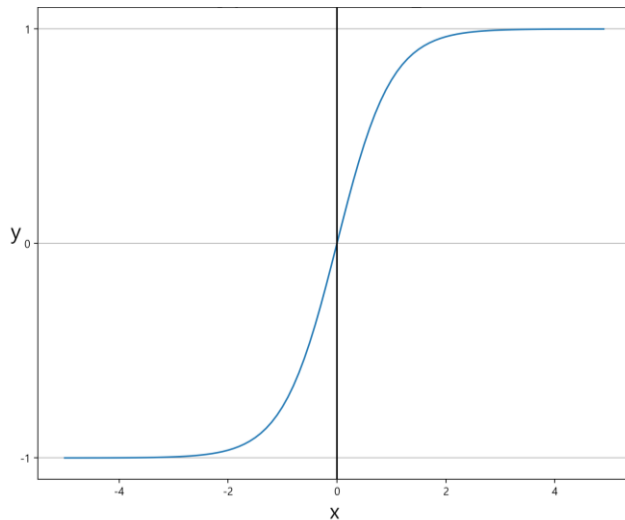
하이퍼볼릭 탄젠트 함수 🔍

■ **-1 ~ 1 사이의 값을 출력**하고, 중앙값은 0

```
def tanh(x):  
    p_exp_x = np.exp(x)  
    m_exp_x = np.exp(-x)  
    y = (p_exp_x - m_exp_x) / (p_exp_x + m_exp_x)  
    return y
```

```
x = 1.0  
y = tanh(x)  
print(y)    # 0.7615941559557649
```

```
x = -1.0  
y = tanh(x)  
print(y)    # -0.7615941559557649
```



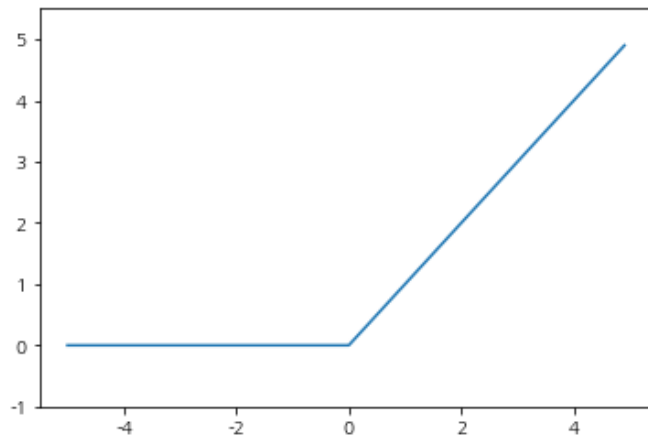


Relu 함수



- 시그모이드 함수는 신경망 분야에 오래전부터 이용해왔지만, 최근에는 ReLU(Rectified Linear Unit, 렐루) 함수를 주로 이용함
 - ↻ 입력이 0을 넘으면 그 입력을 그대로 출력하고, 0 이하이면 0을 출력하는 함수
 - ↻ 수식으로 표현하면 아래와 같고, 이 식을 쉽게 구현해 쓸 수 있음

$$h(x) = \begin{cases} x & (x > 0) \\ 0 & (x \leq 0) \end{cases}$$



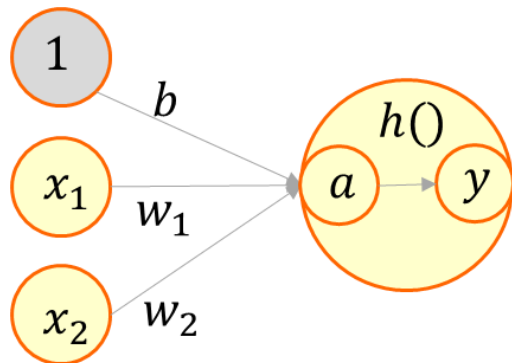


Relu 함수



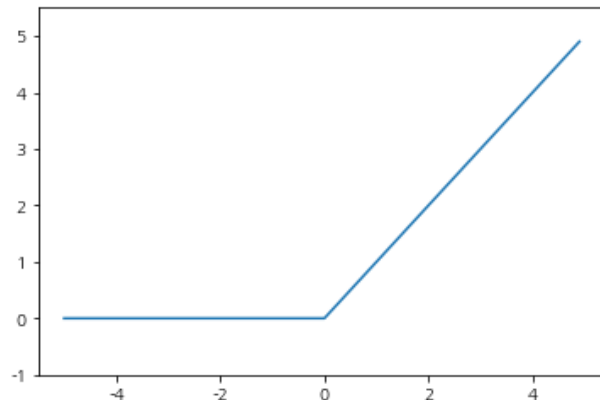
- 신경망에서 활성화 함수로 Relu 함수를 이용하면, 어떠한 값이 들어와도 0을 넘으면 그 입력을 그대로 출력하고, 0 이하이면 0을 출력하여 다음 뉴런에 전달됨

즉, 0을 넘으면 그 입력을 그대로 출력하고,
0 이하이면 0의 값이 다음 뉴런에 전달됨



활성화 함수의 처리 과정

$$h(x) = \begin{cases} x & (x > 0) \\ 0 & (x \leq 0) \end{cases}$$





Relu 함수

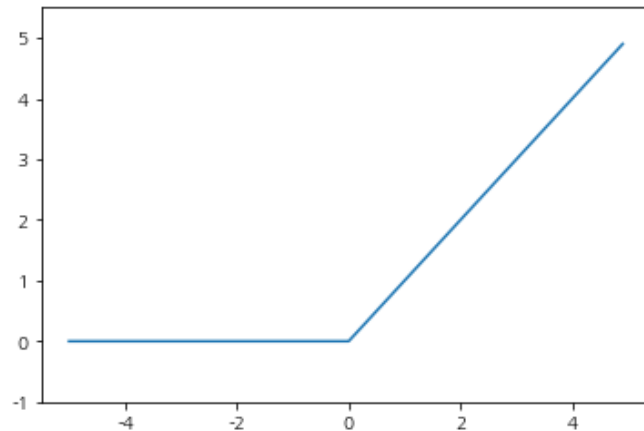


■ 0을 넘으면 그 입력을 그대로 출력하고, 0 이하이면 0의 값을 출력

```
def relu(x):  
    return np.maximum(0, x)
```

```
x = 1.0  
y = relu(x)  
print(y)  # 1.0
```

```
x = -1.0  
y = relu(x)  
print(y)  # 0.0
```



아래 그림과 같이 3층 인공신경망 네트워크를 구성한다고 가정해 보자.

- 활성화 함수, 뉴런 수(입력, 은닉, 출력), 반복회수 등의 하이퍼파라미터는 어떻게 설정해야 할까?
- 사람의 역할? 알고리즘의 역할?

생각해 볼 문제?

